



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Writing a Program

Francesco Guerra
francesco.guerra@unimore.it

Objectives

- Analyze some of the issues involved in producing a simple program:
 - Requirements
 - Design constraints
 - Testing
 - Effort estimation
 - Implementation details
- Understand the sequence of activities involved in writing even a simple program.

A simple problem

Problem

Given a collection of lines of text (strings) stored in a file, sort them in alphabetical order, and write them to another file.

A simple problem

Problem

Given a collection of lines of text (strings) stored in a file, sort them in alphabetical order, and write them to another file

- ✱ A problem statement such the one above does not completely specify the problem
- ✱ Program requirements: statements that define and qualify what the program needs to do
- ✱ Design constraints: statements that constraints the ways in which the software can be designed and implemented

Requirements

- In many business transactions, a requirement is something that absolutely must happen
- In software engineering many items are negotiable
 - Every requirement has a cost, the clients may decide that they do not really need it after they understand the related costs
 - Requirements “needed” and “nice to have”
- Functional requirements: what the program does
 - e.g., sorting a file (with all details specified)
- Nonfunctional requirements: the manner in which the functional requirements need to be achieved
 - e.g., performance, usability, maintainability

Requirements

- Some design constraints are often considered as nonfunctional requirements
 - Program language
 - Specific graphical user interface
 - They can be defined as design constraints if needed by the program to use a specific program / interface to interoperate with other programs
- Requirements are established by the client with the help of the software engineer

Functional Requirements

✿ Input formats

- ✿ What is the format for the input data?
- ✿ How should data be stored?
- ✿ What is a character?
- ✿ What is the line separator?

✿ Sorting

- ✿ Ascending or descending order?
- ✿ How to manage non-alphabetic characters?

✿ Special cases, boundaries and error conditions

- ✿ How should we handle boundary cases such as empty lines / files?

Nonfunctional requirements

■ Performance requirements

- The program needs to finish most or all inputs within a certain amount of time

■ Real-time requirements

- It must complete the processing in a given amount of time
- The variability of the running time is also a big issue

■ Modifiability requirements

- It is important to know
 - the life expectancy of the program
 - If there is a plan to modify the program
- Requirements change over the time
- Even if the modifiability requirements are low, you still need to be able to understand the program for debugging purposes

Nonfunctional requirements / Design constraints

✿ User interface

- ✿ what kind of user interface should the program have?
 - ✿ CLI or GUI?

✿ Typical and maximum input sizes

- ✿ Depending on the typical input sizes, we may want to spend different amount of times on algorithms and performance optimizations

✿ Platforms

- ✿ On which platforms does the program need to run? This decision impacts
 - ✿ Architecture
 - ✿ Operating system
 - ✿ Libraries ...
- ✿ Good programming practices help achieve portability

✿ Schedule requirements

- ✿ The final deadline for completing the project impacts on the costs

✿ Programming Language

- ✿ This is technically a design decision, but it can be considered also as a design constraint

✿ Algorithms

- ✿ There are several aspects that can be influenced by the choice of the algorithm (language / libraries / ...)
- ✿ Performance is usually one of the most important influences in the choice of the algorithm
- ✿ Algorithm can be also considered as design constraints or functional requirements

✿ Software structure

Testing

- Testing must be done while the program is being developed and after it is completed
 - Acceptance testing, testing done by clients (the program runs as specified)
 - Unit testing, a process followed by a programmer to test each piece or unit of software
 - Extreme programming: methodology that says that programmers should write the test cases before writing the code
- Black box testing
 - Test cases are only based on the requirement specifications
- White testing
 - Test cases can be designed while looking at the design and code implementation

Estimating effort

- ✱ It is one of the most important aspects of a software project
 - ✱ It is required to produce a cost estimate and a schedule
 - ✱ This is a difficult task!
- ✱ Consider the “Simple” Problem
- ✱ *Given a collection of lines of text (strings) stored in a file, sort them in alphabetical order, and write them to another file*
- ✱ Answer these questions in class:
 - ✱ How long (in elapsed-time) would it take you to implement this solution?
 - ✱ How much overall effort (in person-hours) will this take?

Estimating effort (2)

- The time required for straightforwardly writing the software program with no interruption
 - Step 1: think about the ideal time to complete it
- Is the assumption made in Step 1 realistic?
 - Can you spend time in this?
 - If you are asked to do it as soon as possible, can you estimated when you would be finished, starting right now?
 - Step 2: calendar time

Estimating effort (3)

- Let us divide the task in several subtasks
 - You create a class *StringSorter* and 3 public methods: Read, Write and Sort
 - The algorithm
 - Finds the latest element
 - Puts it at the end of the array
 - Sort the rest of the array with the same mechanism
 - You create a method called *IndexOfLatest* that return the index of the biggest element in an array

	Ideal time	Calendar Time
IndexOfLatest		
Sort		
Read		
Write		
GUI		
Testing		
Total		

Estimating effort (4)

- Comparing the actual time required with the estimated
 - The estimates in general tend to be more accurate after dividing it in tasks
 - Nevertheless, estimates tend to be inaccurate, even for well-defined tasks
- One big problem is that
 - estimation is performed before the project is done
 - in reality a lot of information is needed to be able to provide a good estimation

Implementation -

A few pointes on implementation

- ✿ To be consistent, in the choice of names, capitalization and programming conventions
 - ✿ Conventions of the programming language (in Java classes start with uppercase letters, variables with lowercase letters)
- ✿ Chose names carefully
 - ✿ Names for functions and variables should be descriptive
 - ✿ Long descriptive names for things that will have a global scope
 - ✿ Short names for local references
- ✿ Test before using a function or method
- ✿ Know the standard library
 - ✿ This library includes common functions, you don't have to reinvent the wheel!
 - ✿ This save extra time and extra work
- ✿ Perform a Review of the code
 - ✿ Showing the code to other people helps to detect errors, inconsistencies and bad naming

Basic Design

- Our basic design separates sorting functionalities from user interface
 - This is a good practice since user interfaces tend change faster than functionalities
- One class (StringSorter) and four methods
 - Reading the strings from a file
 - Sorting the collection of strings
 - Writing the strings to a file
 - Combining the methods, taking the input and output files.
- Unit testing with Junit
 - It uses Java's reflection capabilities to execute testing methods
 - assertEquals to verify if two values that should be equal are truly equal

Implementation

```
1 import java.io.* ; // for Reader (and subclasses) , Writer (and subclasses) and IOException
2 import java.util.* ; // for List , ArrayList , Iterator
3
4 public class StringSorter {
5     ArrayList<String> lines ;
```

- ArrayList: a container that can grow dynamically and support indexed access to its components
 - It is like a vector
 - Lines is not private for the testing

Implementation- Read a Stream

```
9      public void readFromStream( Reader r ) throws IOException
10     {
11         BufferedReader br=new BufferedReader ( r );
12         lines=new ArrayList<String> ( ) ;
13
14         while ( true ) {
15             String input=br . readLine ( ) ;
16             if ( input==null )
17                 break ;
18             lines . add ( input ) ;
19         }
20     }
```

- Reader: for reading text-based streams
- The same method can be used for reading from standard input / network / file

Implementation- Building the sorting functionality

Algorithm

- Find the largest element in the array
- Swap it with the last element
 - Placing the largest element at the end of the array
- Repeat it with the rest of the array

Two functions

- Swap
- Find the index of the largest element

```
46 static void swap ( List<String> l, int i1, int i2 ) {  
47     String tmp=l.get ( i1 );  
48     l.set ( i1, l.get ( i2 ) );  
49     l.set ( i2, tmp );  
50 }
```

Implementation- Building the sorting functionality (2)

```
32  static int findIdxBiggest ( List<String> l, int from, int to) {
33      String biggest=l.get ( 0 );
34      int idxBiggest=from ;
35
36      for ( int i=from+1; i<=to; ++i ) {
37          if ( biggest.compareTo ( l.get ( i ) ) < 0 ) { // it is bigger than biggest
38              biggest= l.get ( i );
39              idxBiggest=i ;
40          }
41      }
42      return idxBiggest ;
43  }
```

```
52  public void sort() {
53      for ( int i=lines.size ( ) - 1 ; i>0 ; --i ) {
54          int big=findIdxBiggest ( lines , 0 , i );
55          swap ( lines , i , big ) ;
56      }
57  }
```

Implementation- Building the sorting functionality (3)

```
60 | void sort ( ) {  
61 |     java . util . Collections . sort ( lines ) ;  
62 | }
```

- ✿ We could have used a much more easier implementation
 - ✿ Sort function provided by the standard java library

Implementation- writing to the file and sort

```
22 public void writeToStream ( Writer w ) throws IOException {
23     PrintWriter pw=new PrintWriter ( w );
24     Iterator i=lines . iterator ( ) ;
25     while ( i . hasNext ( ) ) {
26         pw . println ( ( String ) ( i . next ( ) ) ) ;
27     }
28 }
```

```
65 public void sort ( String inputFileName , String outputFileName ) throws IOException{
66     Reader in=new FileReader ( inputFileName ) ;
67     Writer out=new FileWriter ( outputFileName ) ;
68
69     StringSorter ss=new StringSorter ( ) ;
70     ss . readFromStream ( in ) ;
71     ss . sort ( ) ;
72     ss . writeToStream ( out ) ;
73
74     in . close ( ) ;
75     out . close ( ) ;
76 }
```

User Interfaces

- ✿ In a first implementation we could execute the program via the Command Line
 - ✿ This is not a GUI
 - ✿ For some people this would not be a useful interface
- ✿ If you are accustomed to only using GUIs a CLI is not the best solution
 - ✿ It is really simple to build BAD GUIs
- ✿ How many kinds of interfaces we can develop for the problem on hands?

Summary



A “Simple” Set of Steps

- 1) Understand the problem – requirements
 - **Functionalities**
 - **Non-functionalities**: performance, security, modifiability, marketability, etc.
- 2) Perform Some Design --- based on requirements
 - Organizing the **functionalities** in some sequence; possibly using some diagrams
 - Focus on **input/output** (data, formats, organization)
 - Think about some constraints (non-functionalities) such as **speed**, **UI looks**, **programming language**, **dependencies**, etc.
 - any specific **algorithm** and improvements on sequence of functionalities.

A “Simple” Set of Steps (cont.)

- **3) Code/Implement – Turning the design into actual code**
 - Depending on how much design is completed, one may either directly engage in conversion to code (*language dependent*) or do some more designing.
 - A) Converting input/output to specific UI Interface or I/O format
 - B) Sequencing the processing in the desired order
 - C) Ensuring and converting the processing “algorithm” correctly to the target language construct.
 - D) figure out how to use language library (properly)

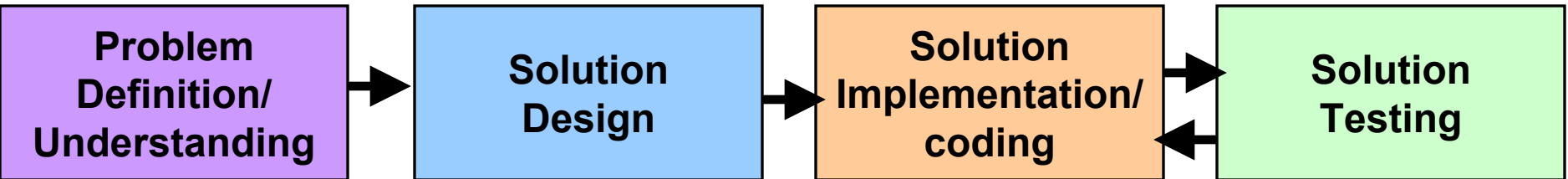
A “Simple” Set of Steps (cont.)

- **4) Validate/Test the program** – check the program results (*via output*) with some predetermined set of inputs.
 - The pre-determined inputs are “test cases” and requires some thinking.
 - If the results do not match what is expected then:
 - “Debug”
 - Fix
 - Retest ---- revalidate
 - **Stop** when all test cases produce the expected results.

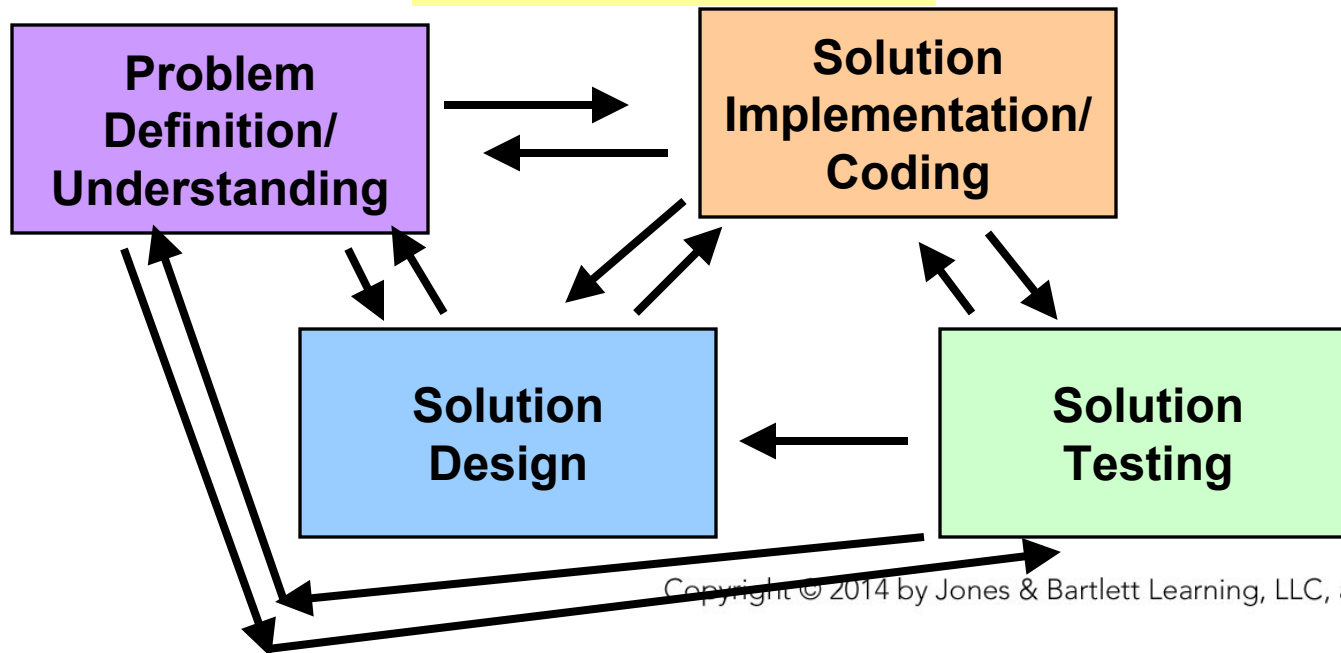
Question ---- **How many test cases should we develop & run ---?**

What Really Happens ?

“Imagined” – Ideal Situation

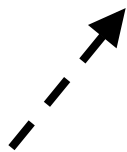


“Actual” - Happening



Code is “Done!” What Else Matters?

- **How Long** (elapsed time) *did it take to complete the work?*
- **How much effort** (total person hours) *is expended to do the work?*
- *Does the solution **solve the complete problem**?*
- **How “good”** *is the work – (code, design, documentation, testing, etc.)?*



based on ?



MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.

Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Fifth Edition), Jones & Bartlett Learning, 2022